

Тестування продуктивності додатку QA Pro REST App

Налаштування середовища та створення тест-плану

Створений «Base test plan» призначений для перевірки продуктивності додатку QA Pro REST App.

«Base test plan» охоплює наступні компоненти:

- **User Defined Variables:** - включає змінні, які використовуються в запитах повторно.

User Defined Variables	
Name:	Value
localhost	localhost
port	3001

- **Запити:**

1. Отримати всіх героїв:

HTTP Request

Name:

Comments:

Basic

Advanced

Web Server

Protocol [http]: Server Name or IP: Port Number:

HTTP Request

GET Content encoding:

2. Отримати героя за id (при цьому id героя визначатиметься випадковим чином):

HTTP Request

Name:

Comments:

Basic

Advanced

Web Server

Protocol [http]: Server Name or IP: Port Number:

HTTP Request

GET Content encoding:

3. Створити героя:

Запит на створення нового героя має тіло запиту з полями:

name — визначається як набір із 10 випадкових символів;

age - визначається як випадкове число від 10 до 80.

HTTP Request

Name: Create new hero

Comments:

Basic Advanced

Web Server

Protocol [http]: http Server Name or IP: \${localhost} Port Number: \${port}

HTTP Request

POST Path: /character Content encoding:

☐ Redirect Automatically ☒ Follow Redirects ☒ Use KeepAlive ☐ Use multipart/form-data ☐ Browser-compatible headers

Parameters Body Data Files Upload

```

1 Body
2 {
3   "name": "${__RandomString(10,abcdefghijklmnoprtsufzwq,)}",
4   "age": "${__Random(10,80,)}"
5 }
6

```

4. Змінити героя (за id — визначається як випадкове число):

В запиті поля name та age визначаються як випадкові значення.

HTTP Request

Name: Change hero

Comments:

Basic Advanced

Web Server

Protocol [http]: http Server Name or IP: \${localhost} Port Number: \${port}

HTTP Request

PUT Path: /character/\${__Random(1,3,)} Content encoding:

☐ Redirect Automatically ☒ Follow Redirects ☒ Use KeepAlive ☐ Use multipart/form-data ☐ Browser-compatible headers

Parameters Body Data Files Upload

```

1 {
2   "name": "${__RandomString(7,abcdefghijklmnoprtsufzwq,)}",
3   "age": "${__Random(20,80,)}"
4 }
5

```

5. Видалити героя (за id — визначається як випадкове число):

HTTP Request

Name: Detele hero

Comments:

Basic Advanced

Web Server

Protocol [http]: http Server Name or IP: \${localhost} Port Number: \${port}

HTTP Request

DELETE Path: /character/\${__Random(1,3,)} Content encoding:

- **Assertions:**

До всіх запитів додано перевірки:

- На статус код:

Response Assertion

Name:

Comments:

Apply to:

☐ Main sample and sub-samples
 ☒ Main sample only
 ☐ Sub-samples only
 ☐ JMeter Variable Name to use

Field to Test

☐ Text Response
 ☒ Response Code
 ☐ Response Message
 ☐ Response Headers

☐ Request Headers
 ☐ URL Sampled
 ☐ Document (text)
 ☐ Ignore Status

☐ Request Data

Pattern Matching Rules

☐ Contains
 ☐ Matches
 ☒ Equals
 ☐ Substring
 ☐ Not
 ☐ Or

Patterns to Test

1 200

- На час виконання запиту (має не перевищувати 30 секунд):

Duration Assertion

Name:

Comments:

Apply to:

☐ Main sample and sub-samples
 ☒ Main sample only
 ☐ Sub-samples only

Duration to Assert

Duration in milliseconds:

- **Listeners:**

Для збору та візуалізації метрик під час виконання запитів та тестів додано наступні елементи:

- **View Results Tree** - дозволяє перевірити весь перелік виконаних запитів, проаналізувати, які запити були вони виконані успішно і побачити тіло відповіді:

View Results Tree

Name:

Comments:

Write results to file / Read from file

Filename

Search: ☐ Case sensitive ☐ Regular exp.

Text

- ✓ Get all heros
- ✓ Get hero by id
- ✓ Create new hero
- ✓ Change hero
- ✓ Detele hero
- ✓ Get all heros
- ✓ Get hero by id

Sampler result Request Response data

Response Body Response headers

```
[{"name":"qwe","age":3},{"name":"Hello world","age":12},{"name":"privet","age":22}]
```

- **Aggregate Report** — надає статистичну інформацію в цілому по «Base Test plan».

Aggregate Report

Name:

Aggregate Report

Comments:

Write results to file / Read from file

Filename

/home/ivakula/Documents/AWork/Jmeter/apache-jmeter-5.6.3/bin/myProject/report.csv

Browse...

Log/Display Only: ☐ Errors ☐ Successes

Label	# Samples	Average	Median	90% Line	95% Line	99% Line	Min	Maximum	Error %	Throughput	Received KB/sec
Get all heros	1000	2011	2127	3013	3258	3288	10	3339	0.00%	235.0/sec	57.64
Get hero by id	1000	7	7	13	15	23	2	39	0.00%	235.3/sec	54.00
Create new h...	1000	4	4	8	9	15	1	39	0.00%	235.7/sec	54.10
Change hero	1000	3	3	7	9	14	1	29	0.00%	235.9/sec	54.14
Detete hero	1000	4	3	7	9	15	1	29	0.00%	236.1/sec	57.40
TOTAL	5000	406	5	2127	2765	3258	1	3339	0.00%	1171.5/sec	275.76

З даного звіту видно, що при відсутності навантаження (1000 користувачів) максимальний час виконання запиту становить трохи більше 3 секунди.

При цьому з таблиці видно, що найдовше обробляється запит «Get all heroes». 99% запитів «Get all heroes» виконуються швидше, ніж за 3288 мілісекунд.

Всі запити (тобто 100%) виконані успішно.

- **Response Time Over Time** - дані будуть проаналізовані при використанні різних профілів навантаження.
- **Perfmon Metrics Collector** - дані будуть проаналізовані при використанні різних профілів навантаження.
- **Active Threads Over Time** - дані будуть проаналізовані при використанні різних профілів навантаження.
- **Synthesis Report** - дані будуть проаналізовані при використанні різних профілів навантаження.

Кореляція та моніторинг ресурсів

Для того, щоб проаналізувати продуктивність додатку при різних навантаженнях, створимо різні Threads (відповідатимуть різним профілям навантаження) в різних налаштуваннях.

1. Для проведення Load testing створюємо **Stepping Thread Group** з наступними налаштуваннями:

jp@gc - Stepping Thread Group

Name: jp@gc - Stepping Thread Group

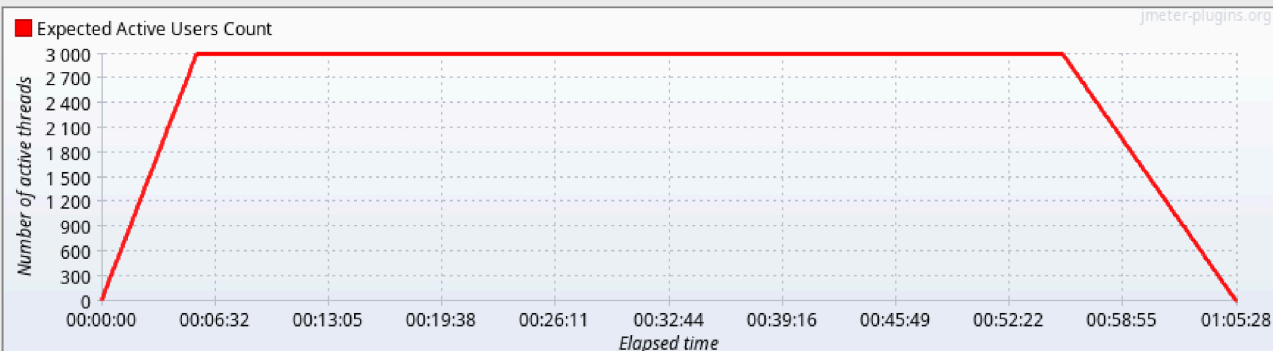
Comments:

Action to be taken after a Sampler error

☒ Continue ☐ Start Next Thread Loop ☐ Stop Thread ☐ Stop Test ☐ Stop Test Now

Threads Scheduling Parameters

This group will start threads:
First, wait for seconds;
Then start threads;
Next, add threads every seconds,
using ramp-up seconds.
Then hold load for seconds.
Finally, stop threads every seconds.



Вибираємо наступний сценарій:

Цільове навантаження — 3000 користувачів.

Очікування після початку тесту — 0 секунд.

100 потоків додаються кожну секунду з наростанням (або часом крокового переходу) 10 секунд.

Цільове навантаження утримується протягом 3000 секунд

Нарешті, 5 потоків зупиняються кожну секунду.

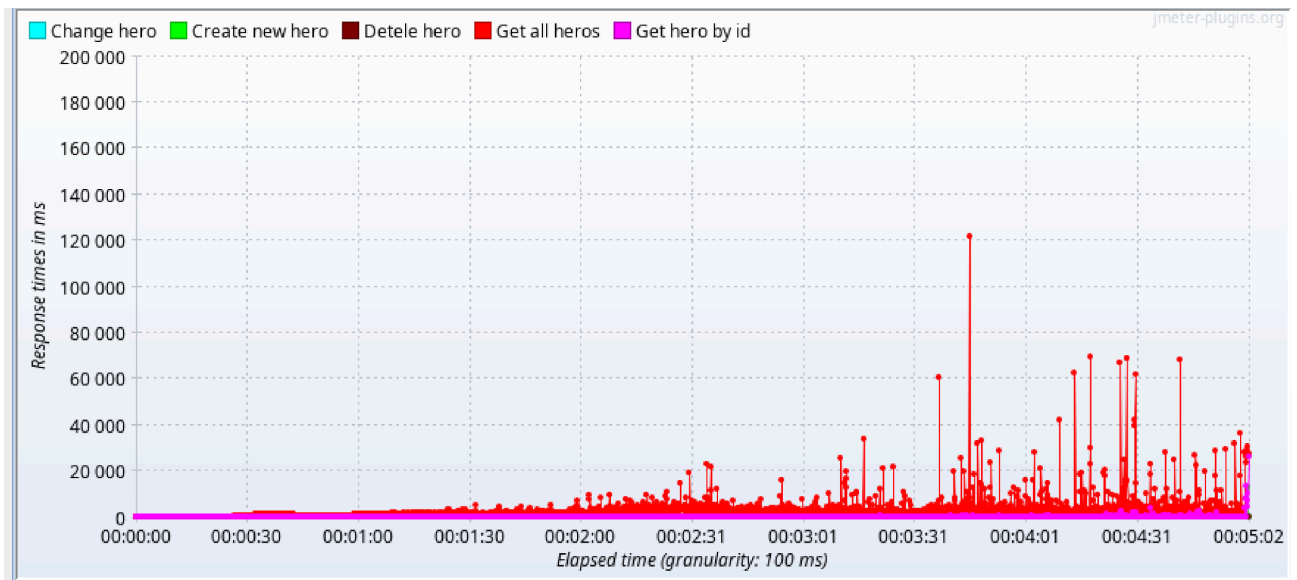
Тест починається одразу після запуску, оскільки очікування триває 0 секунд.

Кожну секунду буде додаватися 100 користувачів, поки ми не досягнемо 3000 користувачів.

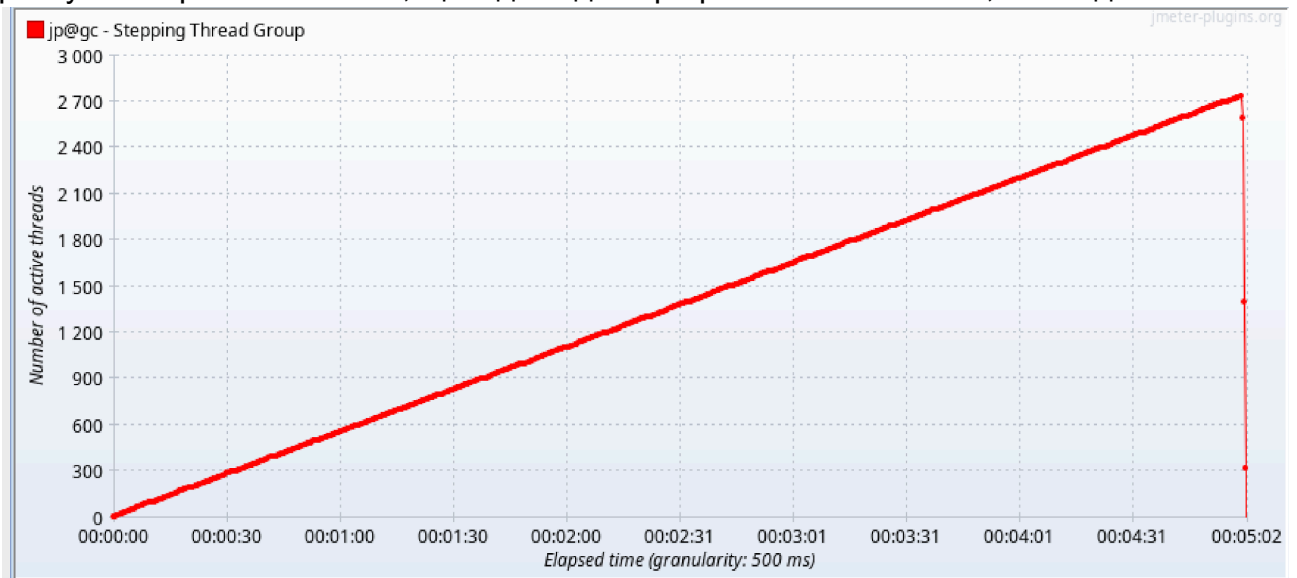
Для виконання кожного кроку (зі 100 користувачами) знадобиться 1 секунда. Після цього JMeter чекає 10 секунд перед початком наступного кроку.

Після досягнення 3000 потоків усі вони продовжуватимуть працювати разом на сервері протягом 3 хвилин.

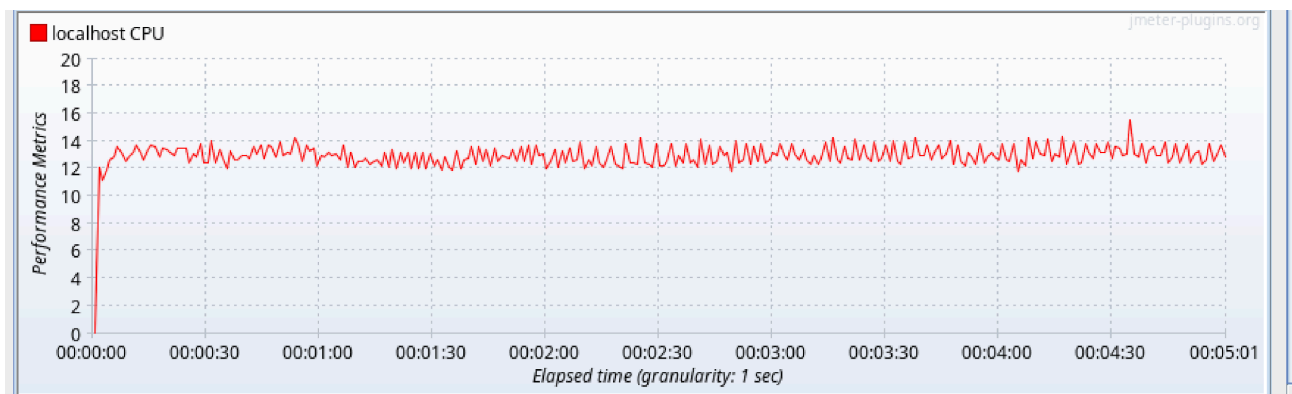
Зі звіту **Response Time Over Time** видно, що середній час відповіді на всі запити, крім Get all heroes, з часом не змінюється. В той час як час відповіді на запит Get all heroes починає з часом зростати, що свідчить про збільшення обсягів даних. Крім того, спостерігаються окремі сплески значень, що потребують окремого аналізу.

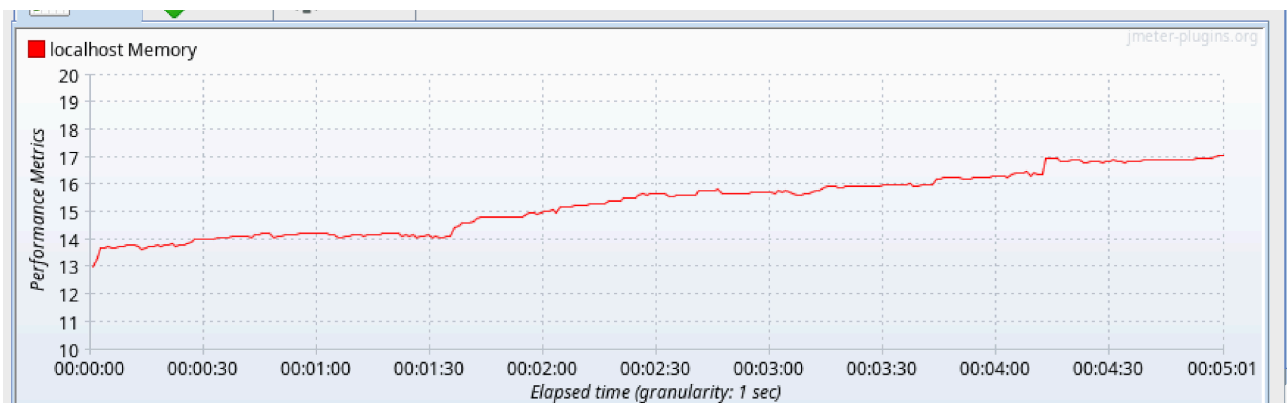


Зі звіту **Active Threads Over Time** видно, що загальна кількість активних користувачів зростає з часом, що відповідає профілю навантаження, яке задали.



Звіти **Perfmon Metrics Collector** дозволяють проаналізувати завантаженість оперативної пам'яті (поступово збільшується з часом, але становить не більше 18%) та процесору (в середньому не змінюється і становить 12-14%).





Зі звіту **Synthesis Report** видно, що з часом при збільшенні навантаження на систему з'являються запити, що виконуються з помилкою, що свідчить про те, що сервер не завжди справляється з існуючим навантаженням.

jp@gc - Synthesis Report (filtered)

Name:

Comments:

Write results to file / Read from file

Filename Log/Display Only: ☐ Errors ☐ Successes

Filter settings

Start / End offset (sec)	Include labels	RegExp	Exclude labels	RegExp
	☐		☐	

[Help on this plugin](#)

Label	# Samples	Average	Min	Max	90% Line	Std. Dev.	Error %	Throughput	Received K...	Avg. Bytes
Get all heros	113339	3577	2	153293	5156	10895.03	3.57%	375.2/sec	182.85	499.0
Get hero b...	110723	55	1	96787	19	908.64	0.10%	366.7/sec	84.87	237.0
Create new...	110636	9	0	26421	20	188.38	0.01%	366.4/sec	84.16	235.2
Change hero	110608	7	0	321	22	13.17	0.00%	366.3/sec	84.10	235.1
Detele hero	110598	7	0	323	23	13.87	0.00%	366.3/sec	89.09	249.0
TOTAL	555904	745	0	153293	1322	5140.73	0.75%	1840.3/sec	524.92	292.1

При збільшенні числа користувачів до 7000 та додаванні 500 нових користувачів кожної секунди на 4й хвилині проходження випробування вимкнувся JMeter. Це свідчить про те, що комп'ютер, на якому працював додаток, і який виконував водночас функцію сервера, не витримав навантаження. Для забезпечення стабільної роботи застосунку необхідно мати більш потужний комп'ютер або зменшувати кількість користувачів, що одночасно працюють з додатком, можна, крім того, збільшити показник Ramp-up.

2. Для проведення Stress testing створюємо **Concurrency Thread Group** з наступними налаштуваннями: Кожні 2 хвилини буде додаватися по 6 потоків, доки не буде досягнуто 30 потоків. Після досягнення 30 потоків, протягом 10 хвилин навантажуватиметься сервер.

На відміну від стандартної групи потоків, всі потоки не будуть створені заздалегідь, тому додаткова пам'ять не використовуватиметься. Це гарна заміна для Stepping Thread Group, оскільки вона дозволяє потокам витончено завершувати свою роботу.

bzm - Concurrency Thread Group

Name: bzm - Concurrency Thread Group

Comments:

Action to be taken after a Sampler error

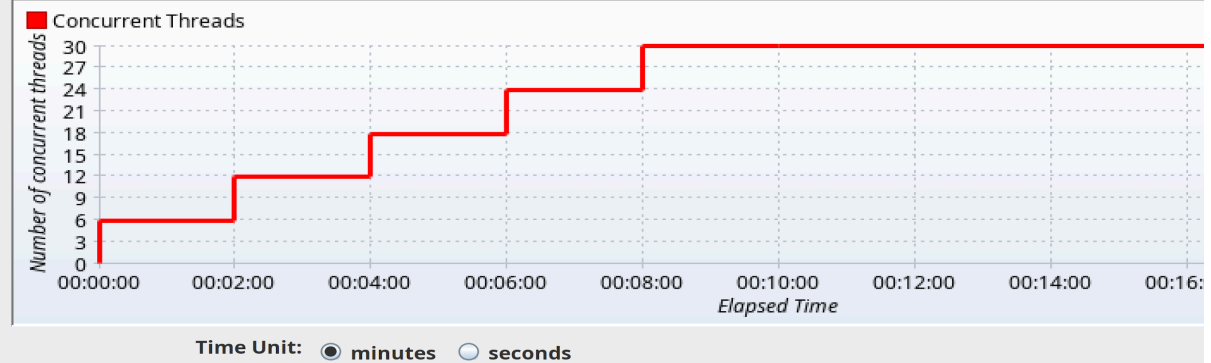
☒ Continue ☐ Start Next Thread Loop ☐ Stop Thread ☐ Stop Test ☐ Stop Test Now

Target Concurrency: 30

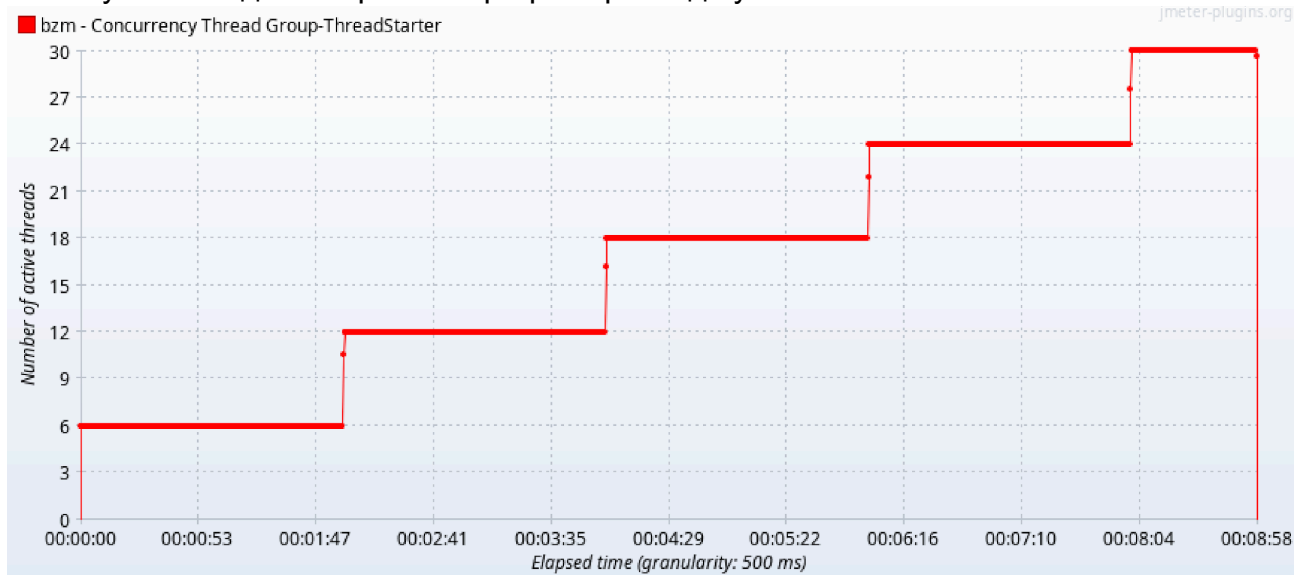
Ramp Up Time (min): 10

Ramp-Up Steps Count: 5

Hold Target Rate Time (min): 10



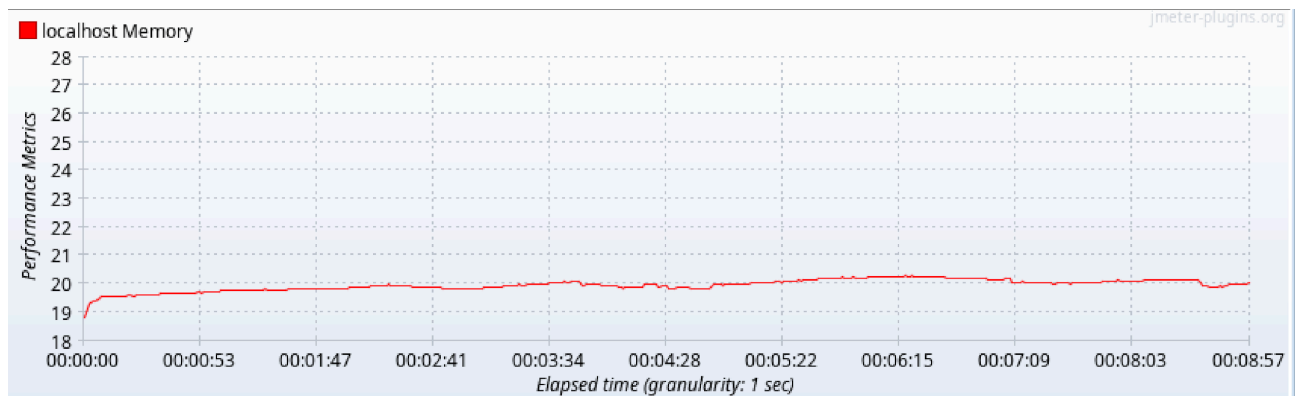
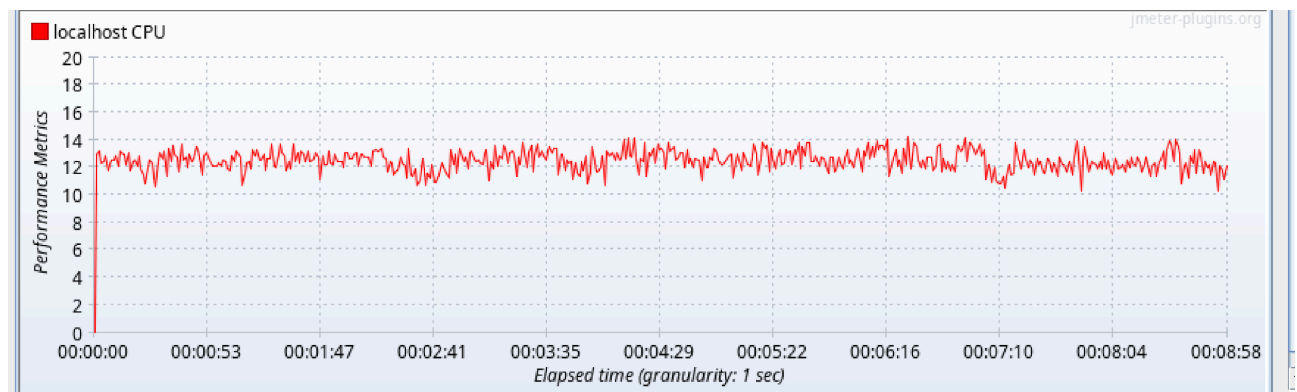
З графіка звіту Active Threads Over Time видно, що кількість активних потоків збільшується згідно вибраного профілю розподілу:



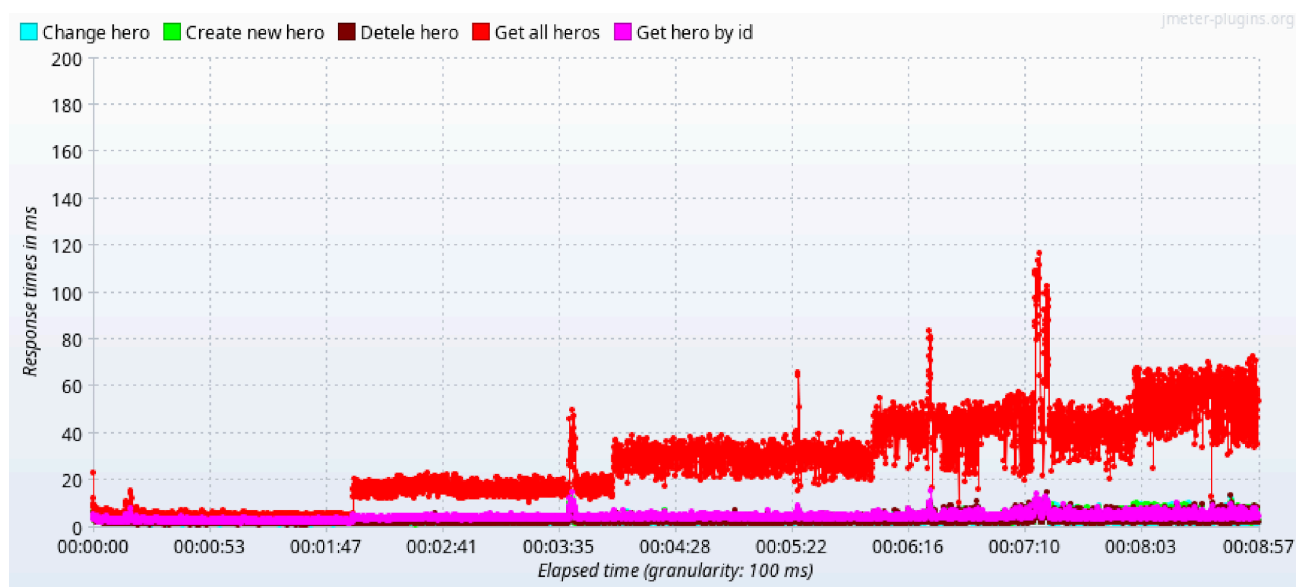
З таблиці Synthesis Report видно, що всі запити успішно виконані (Error=0%).

Help on this plugin										
Label	# Samples	Average	Min	Max	90% Line	Std. Dev.	Error %	Throughput	Received K...	Avg. Bytes
Get all heros	213039	26	1	130	52	18.38	0.00%	396.3/sec	735.86	1901.3
Get hero b...	213013	4	0	33	6	1.91	0.00%	396.3/sec	90.95	235.0
Create new...	213011	2	0	32	5	2.03	0.00%	396.3/sec	90.95	235.0
Change hero	213011	2	0	43	4	1.94	0.00%	396.3/sec	90.95	235.0
Detele hero	213010	2	0	32	4	1.98	0.00%	396.3/sec	96.37	249.0
TOTAL	1065084	7	0	130	24	12.69	0.00%	1981.4/sec	1105.05	571.1

Тако треба зауважити, що завантаження процесору є в середньому стабільне, в той час як завантаження пам'яті поступово з часом зростає.



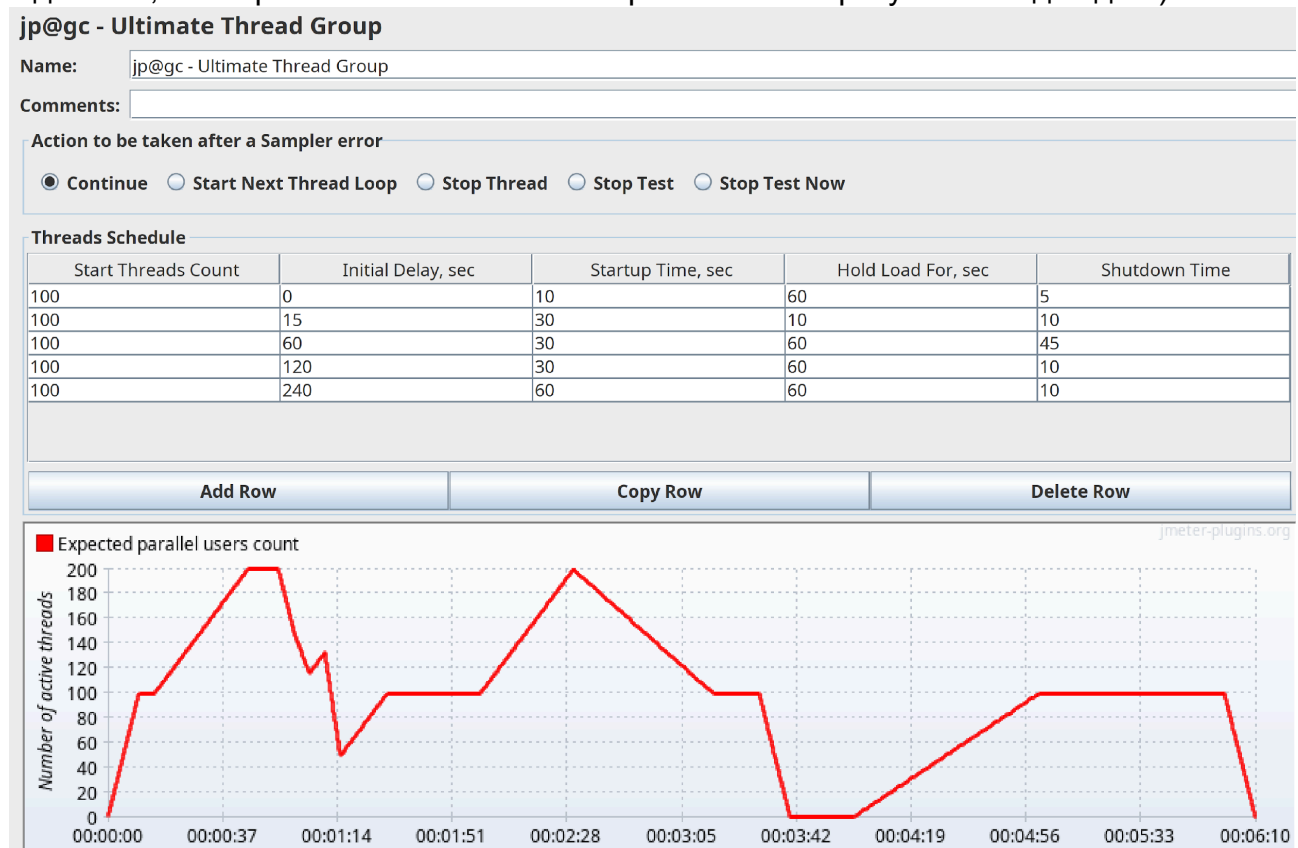
Окремо треба зауважити, що зі зростанням кількості потоків пропорційно зростає час відповіді сервера (Response Times Over Time). Крім того, значно зростає різниця між максимальним та мінімальним часом відповіді. Більш того, з'являються на графіку окремі пікові значення, що не є характерними для загальної моделі. Все це може свідчити про необхідність оптимізації роботи сервера.



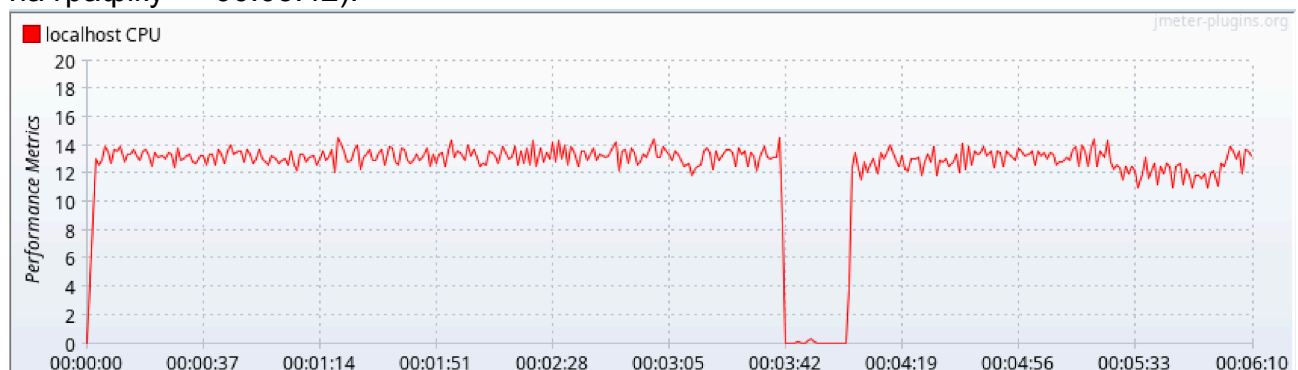
3. Використання **Ultimate Thread Group** дозволяє більш гнучко налаштувати профіль навантаження, врахувавши наступні показники:

- кількість користувачів;
- затримку перед запуском групи користувачів;
- час розгону;
- час затримки на піку;
- час завершення роботи групи користувачів.

Припустимо, що очікується, що система відповідає поведінці користувачів саме з налаштуваннями профілю, що вказані на рисунку (спостерігатиметься два піки навантаження з подальшим різким зменшенням навантаження та відносно пологим підйомом, потім рівень навантаження зберігається та в результаті падає до 0):

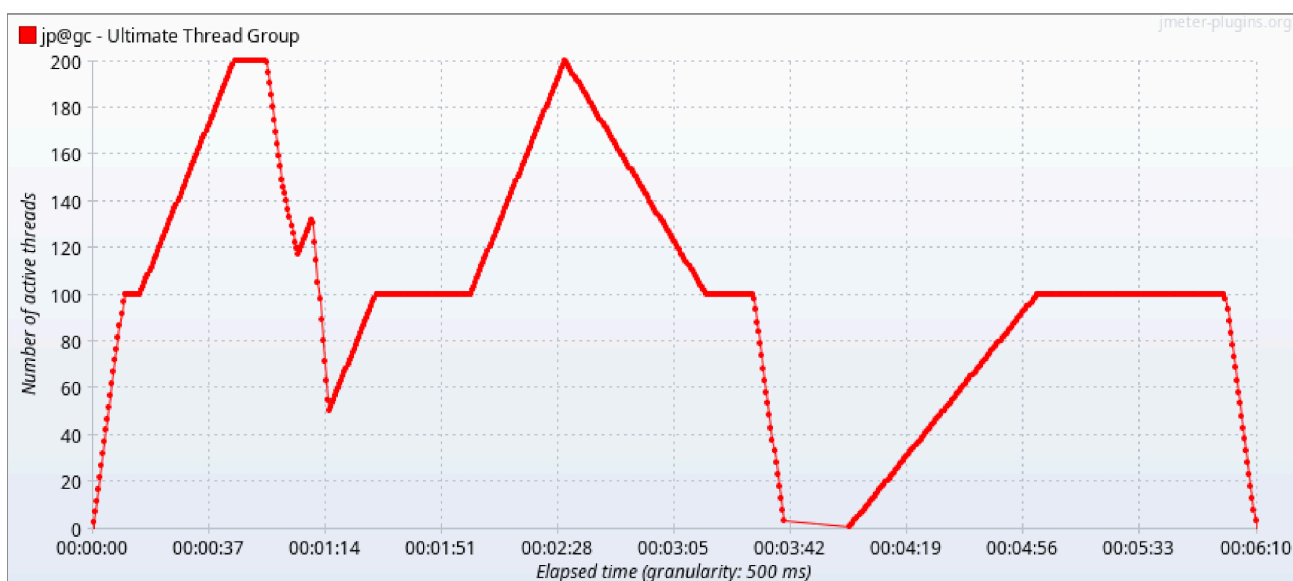


Треба зазначити, що система справляється добре з подібним навантаженням. Процесор та пам'ять завантажені приблизно на 12-14% та 20% відповідно. При зменшенні кількості користувачів різко зменшується і навантаження на систему (час на графіку — 00:03:42):

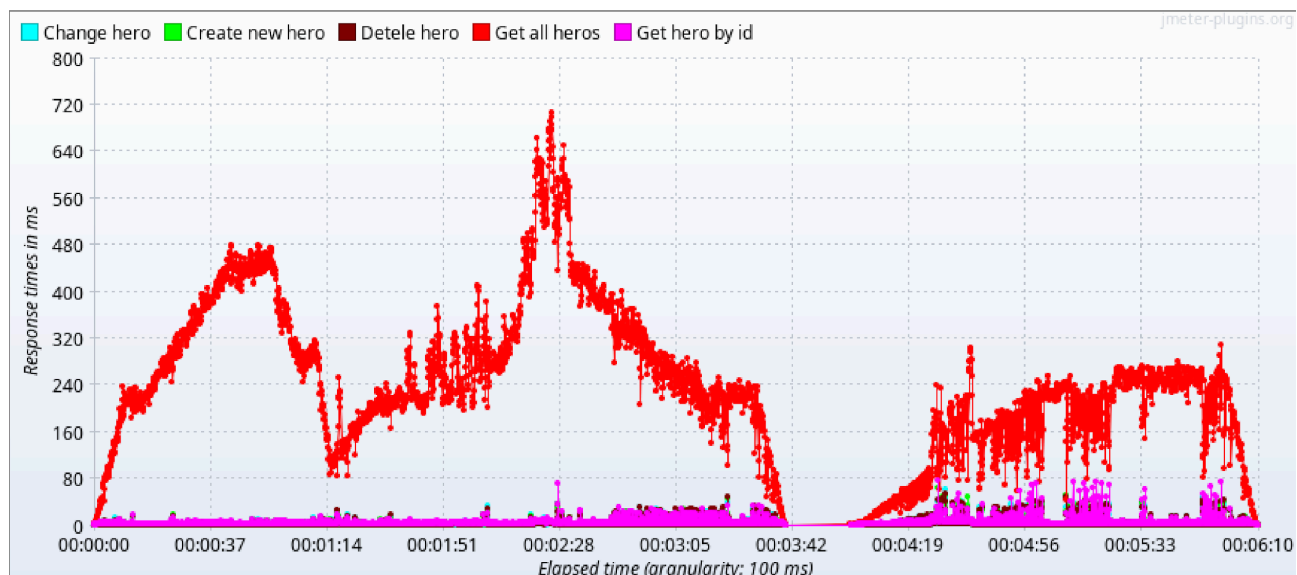




Графік кількості активних користувачів (Active Threads Over Time) повністю відповідає обраній моделі.



Цікавими є дані звіту Response Times Over Time. З графіка видно, що час відповіді збільшується пропорційно кількості користувачів тільки для запита Get all heroes. При різкому збільшенні кількості користувачів час відповіді також різко збільшується (тобто система працює значно повільніше), а при різкому зменшенні кількості користувачів, час відповіді також зменшується (що свідчить про те, що система без проблем повертається в вихідне положення). Для всіх інших запитів час відповіді залишається відносно незначним та стабільним.



Проаналізувавши особливості роботи додатку в розрізі різних профілів навантаження можна зробити висновок, що система коректно працює лише з кількістю користувачів одночасно, що не перевищує 5000. При збільшенні кількості користувачів до 7000 сервер «падає».

Все це вказує на необхідність збільшення потужностей сервера, якщо передбачається, що кількість користувачів, які одночасно звертаються до системи, буде більшою за 5000.

Подальші рекомендації щодо поліпшення роботи програмного забезпечення можливі лише за наявності реальних даних та документації з вказаними вимогами (наприклад, бажаного часу відповіді чи запланованої кількості користувачів), які для навчальної бази відсутні.